FreEQ

Filter modes and processing cost of one six-band equaliser, realised twice.

FreEQ is a six-band equaliser with a choice of minimum phase, linear phase, or a zero-latency analogue-matched mode. The same specification was built twice: once in Rust, using OpenPlugin and GoodLookin, and once in C++, using JUCE. The C++ program was compiled against JUCE 8.0.17-security and again against JUCE 9.0.5. This paper describes how the three filters are made, how their processing cost was measured, and what those measurements show. A +6 dB bell at 1 kHz reads 6.00 dB in minimum phase and in linear phase, and 6.03 dB in the analogue-matched mode, in both realisations.

0.12%	0.50%	one core
Minimum phase, full patch	JUCE linear phase or ZL	100 JUCE FIR equalisers

Figures are the share of one processor needed to play the signal in real time. The spectrum display was not running. Each program finished before the next began.

The instrument

Figure 1 and Figure 2 show the two editors. Both are drawn at 980 by 640 points and keep that shape when the window is resized. Six numbered bands sit under the curve. Each band can be a peak, a high-pass, a low-pass, a low shelf, or a high shelf. MP, LP, and ZL choose minimum phase, linear phase, and the analogue-matched mode. The yellow line is the calculated response. The colour behind it is a separate spectrum display, not part of the processing-cost test.

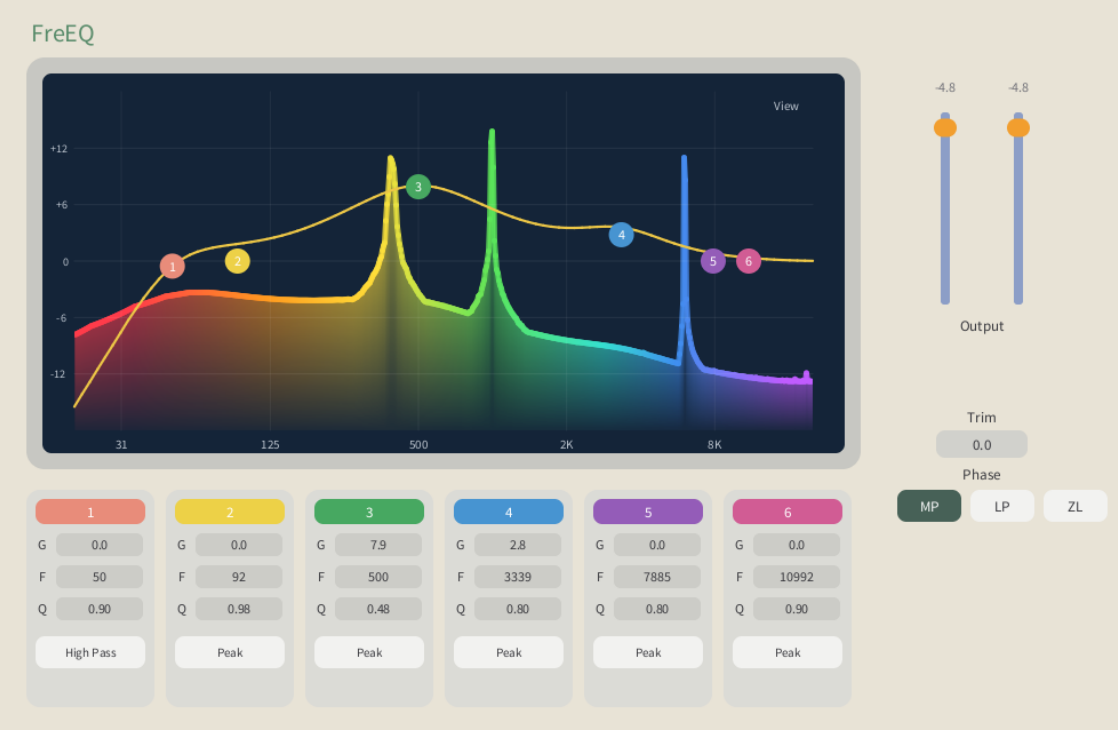


Figure 1. OpenPlugin realisation. Minimum phase is selected. The coloured fill is a 4096-point spectrum of the signal leaving the equaliser.



Figure 2. JUCE realisation. The 8.0.17 and 9.0.5 builds draw the same editor. Analog ZL is selected, and band 3 is a +6 dB bell at 1 kHz.

Filter design

Each of the six bands is one second-order section, the familiar audio biquad: the current sample and two previous input and output samples, combined with five coefficients. The coefficients follow the Bristow-Johnson formulas. A peak raises or lowers a region around a chosen frequency. A low shelf and a high shelf tilt everything below or above a corner. A high-pass and a low-pass remove one side of the spectrum. The high-pass and the low-pass do not use the gain control, and each is a single section, so the slope is 12 dB per octave rather than a choice of 6, 12, or 24. Frequency runs from 20 Hz to 20 kHz on a logarithmic scale. Resonance is set by Q.

Minimum phase. The six sections run one after another on every sample, then a final output gain is applied. Nothing is delayed on purpose. The delay that does occur comes from the filters themselves and changes with frequency: a bell of +6 dB at 1 kHz holds that region back by about eleven samples, while frequencies far from the bell pass almost at once. That is the ordinary sound of an analogue equaliser. The reported latency is zero. A peak whose gain is exactly 0 dB is skipped, because that section is a straight wire. A shelf, a high-pass, or a low-pass is always calculated. The output meter reads the loudest sample in the block once, rather than taking a logarithm of every sample.

Linear phase. The magnitude of the six-section equaliser is measured at the playback sample rate, at 1024 evenly spaced frequencies. The phase of that measurement is then replaced by zero at every frequency. An inverse transform turns the result into an impulse that is symmetric around its centre: the same ringing appears before the peak as after it. The impulse is rotated so the peak lies on sample 512. Every frequency is then delayed by exactly 512 samples, which is 10.7 ms at 48 kHz, and that is the latency the plug-in reports. Playback is overlap-save convolution: each block is transformed at 2048 points, multiplied by the transform of the impulse, and transformed back. Only the new samples are kept. This is the same filter as a 1024-tap walk, without visiting every coefficient on every sample. The pre-ring is the price of a delay that does not depend on frequency.

Analog ZL. A biquad designed at the playback rate is a bilinear copy of an analogue filter. The copy squeezes the frequency axis, gently at low frequencies and strongly near half the sample rate, so the top octave no longer follows the analogue curve. Analog ZL avoids that squeeze by designing the six sections as though the sample rate were sixteen times higher. The magnitude of that high-rate design is then read at the frequencies a 1024-point transform needs at the real playback rate, and transformed back to an impulse. The half of the impulse that would have sounded before time zero is discarded, and the last 64 samples of what remains are faded out. The peak now sits at the start, so the reported latency is zero. The filter is causal, and therefore its phase is not linear: it still delays some frequencies more than others, much as minimum phase does. What has changed is the magnitude, which stays closer to the analogue prototype through the top octave. Playback uses the same overlap-save convolution as linear phase. Removing the pre-ring does not shorten the transform, so it does not reduce the cost.

The picture behind the curve is a different transform. It is a 4096-point analysis of the signal after the equaliser, with a Hann window, a gentle 3 dB-per-octave tilt, and a fall of 24 dB per second. It is drawn for the eye and was switched off for every timing run. The transforms used to design the linear-phase and Analog ZL kernels run only when the settings change, not on every block, so they are outside the timed region as well.

Realisations

Piece	Version	Commit	Date
OpenPlugin SDK	0.1.1-beta.1	8ab9e16	11 Nov 2026
GoodLookin DRI	0.1.0	d8311fa	25 Oct 2026
JUCE 8, EQ build	8.0.17-security	dabd5ad	2 Nov 2026
JUCE 9, EQ build	9.0.5	d60b7a7	2 Nov 2026
JUCE, before	8.0.4	51d11a2	18 Nov 2024

The Rust program uses OpenPlugin 0.1.1-beta.1 and GoodLookin 0.1.0. The C++ program is one set of sources, linked once with JUCE 8.0.17-security and once with the official JUCE 9.0.5. An earlier measurement had used JUCE 8.0.4, from November 2024. Both builds measured here were release builds. Two other copies of JUCE used by a different project carry three local changes on top of 9.0.5. Those copies were not used for the numbers below.

Measurement

All runs used 48 kHz, stereo, and blocks of 512 samples. The stimulus was a 1 kHz sine at amplitude 0.25, with the phase carried from one block into the next so the tone did not click. The clock was the processor time charged to the program, not the time on the wall. After the equaliser loop, the same blocks were copied again with no filtering, and that copying time was subtracted. The reported figure is the remaining time divided by the length of the audio, in percent. One hundred percent means the equaliser alone fills one processor core.

A single equaliser was warmed for 187 blocks, about two seconds, which is not included in the result. The timed region was the next 750 blocks, exactly eight seconds. The whole pass was repeated six times. The first pass was discarded and the median of the other five was kept. For one hundred equalisers the audio length stayed eight seconds, the warmup was the same, and the median was taken from three passes after one discarded pass. Each of the hundred had its own filter state and was given a fresh copy of the block. The signal did not pass through them in series. The three programs ran one after another: OpenPlugin, then JUCE 8.0.17-security, then JUCE 9.0.5. The hundred-equaliser test used that same order.

Three settings were used. Idle sets every band to a peak with 0 dB of gain. One band keeps that arrangement and raises band 3 by 6 dB at 1 kHz with Q equal to 1. Full makes band 1 a high-pass at 100 Hz, band 2 a low-pass at 8 kHz, and bands 3 to 6 peaks of +6 dB with Q equal to 2. The hundred-equaliser test used only the full setting. Output trim was 0 dB throughout. The spectrum display was disabled before the warmup.

Results

Table 1 gives the cost of one stereo equaliser, as a percentage of realtime. Latency is in samples at 48 kHz.

DSP	Load	OpenPlugin	JUCE 8.0.17	JUCE 9.0.5	Latency
Minimum	idle	0.012	0.025	0.021	0
Minimum	one band	0.070	0.104	0.080	0
Minimum	full	0.115	0.229	0.227	0
Linear	idle	1.12	0.51	0.48	512
Linear	one band	1.09	0.51	0.50	512
Linear	full	1.10	0.51	0.50	512
Analog ZL	idle	1.09	0.51	0.50	0
Analog ZL	one band	1.09	0.51	0.50	0
Analog ZL	full	1.09	0.51	0.50	0

Table 2 repeats the full setting with one hundred independent equalisers. One hundred percent is one core. The earlier per-sample convolution needed about nine cores for the two FIR rows.

DSP	OpenPlugin	JUCE 8.0.17	JUCE 9.0.5	Latency each
Minimum	11.3	18.9	18.7	0
Linear	110	44.0	42.6	512
Analog ZL	110	44.1	42.8	0

Discussion

Minimum phase now follows how many sections are actually doing work. Idle, with every peak at 0 dB, is a few hundredths of a percent. One bell is in between. The full patch, where all six sections run, is 0.115% in OpenPlugin and about 0.23% in JUCE. The FIR rows do not move with the patch, because playback is one 2048-point overlap-save of a kernel that is always 1024 taps long. JUCE does that transform with its own FFT, which is why a linear-phase or Analog ZL equaliser costs about 0.50% there and about 1.10% in OpenPlugin.

Table 2 is close to one hundred times Table 1. One hundred full minimum-phase equalisers take 11% of one core in OpenPlugin and about 19% in JUCE. One hundred linear-phase or Analog ZL equalisers take 43 to 44% of one core in JUCE, and just over one core, about 110%, in OpenPlugin. Before this change the same hundred FIR equalisers needed about nine cores. The two JUCE versions still do not separate.

Linear phase and Analog ZL are still chosen for their sound and their delay. Linear phase delays the whole signal by 10.7 ms and rings evenly before and after a transient. Analog ZL keeps the delay at zero and follows the analogue magnitude more closely above a few kilohertz, and it still has the ordinary frequency-dependent delay of a causal filter. They now cost about half of one percent each in JUCE, and about one percent in OpenPlugin, instead of nine. A hundred of them fit in one core on JUCE, and in a little more than one core on OpenPlugin.